
Test instruction

Welcome to the main exam of the course Multi-Agent Systems (XB_0024). This exam will test your knowledge and understanding of the material provided to you and presented in the lectures, the book of Blackburn, Bos and Striegnitz *Learn Prolog Now!* (Chapters 1 to 6, 10 and 11), and the MARBEL Guide. It is not allowed to use materials such as books, scientific papers or slides during the exam. You will have 2:45 hours to complete the exam.

The following tools are permitted:

- Pen and drafting paper

For questions about the content of the test you can contact the teacher. Information about the review of the exam is provided via Canvas.

If you have **not** signed up for this exam, you will not receive a result. Through VUnet you can object to the fact that you can no longer sign up after the expiry of the registration deadline (and the fact that you will not receive a result for this exam). Submit your appeal online within one week after the exam. More information can be found at www.vu.nl/intekenen.

Good luck!

Welkom bij het examen van het vak Multi-Agent Systems (XB_0024). Deze toets test je kennis en begrip van de stof als gepresenteerd in de colleges, het boek van Blackburn, Bos en Striegnitz *Learn Prolog Now!* (Hoofdstukken 1 tot 6, 10 en 11), en de MARBEL Guide. Het is niet toegestaan om hulpmiddelen zoals boeken, publicaties en slides te gebruiken tijdens de toets. Je hebt 2:45 uur om de toets te maken.

Bij deze toets mag je gebruik maken van de volgende hulpmiddelen:

- Pen en kladpapier

Voor vragen over de inhoud van de toets kun je bij de docent terecht. Informatie over de toetsinzage wordt via Canvas verstrekt.

Als je **niet** hebt ingetekend voor dit tentamen, ontvang je geen resultaat. Via VUnet kun je bezwaar maken tegen het feit dat je niet meer kunt intekenen na het verstrijken van de intekendeadline (en je dus geen resultaat krijgt voor dit tentamen). Dien je bezwaarschrift online in binnen één week na het tentamen. Meer informatie vind je op www.vu.nl/intekenen.

Veel succes!

Prolog queries

Question order: Random

The following questions each give a prolog query, after which you are asked to fill in the outcome that Prolog will give based on the query.

Example:

?- $X = a$.

$X = a$.

Question 1 – 21_22_prolog_query5 – 313788.1.0

?- $X+Y = 23+8$.

X = 23, Y = 8.

X=23, Y=8.

X=23,Y=8.

X=23, Y = 8.

X = 23, Y=8.

X = 23, Y = 8

X=23, Y=8

X=23,Y=8

X=23, Y = 8

X = 23, Y=8

Question 2 – 21_22_prolog_query7 – 313829.1.0

?- X is 15+15.

X = 30.

X= 30.

X =30.

X=30.

X = 30

X= 30

X =30

X=30

Question 3 – 21_22_prolog_query8 – 313833.1.0

?- $c(A,a) = c(b,B)$.

A = b, B = a.

A=b, B=a.

A = b, B=a.

A=b, B = a.

B = a, A = b.

A = b, B = a

A=b, B=a

A = b, B=a

A=b, B = a

B = a, A = b

Question 4 – 22-23_prolog_query1 – 348391.2.0

?- X+2 is 5.

false.

False.

Error.

false

False

Error

Question 5 – 22-23_prolog_query2 – 348394.2.0

?- not(A=4) .

false.

False.

Error.

false

False

Error

Prolog Search

Question order: Fixed

The questions in this block will be based on the following knowledge base:

$a(X) :- x(X), y(X), z(X).$

$x(n).$

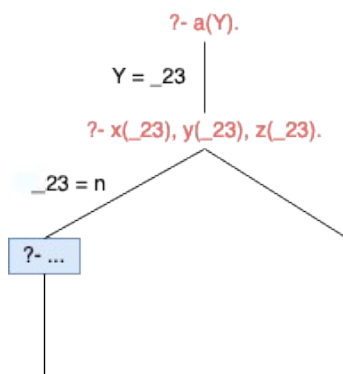
$x(p).$

$y(n).$

$y(p).$

$z(p).$

Part of the search tree for the query ?- $a(Y).$ is given:



Tip: keep this block introduction open when answering the following questions, by clicking the 'show block intro' button above the question.

Question 6 – Prolog search resit 1 – 232370.3.0

Specify the query that prolog will try to match in the left-hand node (the blue box).

?- y(n), z(n).

y(n), z(n).

?- y(n),z(n).

y(n),z(n).

?- y(n), z(n) .

y(n), z(n) .

?- y(n),z(n) .

y(n),z(n) .

Question 7 – Prolog search resit 2 – 232371.2.0

What will be the outcome when the left-hand search branch is completely followed?

A Failure

B Succes

C Infinite

Question 8 – Prolog search resit 3 – 232527.1.1

If the goal in the left-hand branch cannot be further matched with any rule, prolog will try rules that were not explored yet at the point in the tree where this left-hand branch was started. What is the name of this principle of prolog search?

A Backtracking

B Backward chaining

C Floundering

D Occur's check

Question 9 – prolog search 4 – 349083.1.1

Any variable instantiation is specified alongside a search branch when applicable. Give the variable instantiation that applies to the right-hand branch in the given search tree, if any. Otherwise, answer with a cross ('x').

?- _23 = p.

_23 = p.

_23 = p .

?- _23 = p .

Prolog concepts

Question order: Fixed

The following questions address your knowledge of prolog concepts. Answers are either given in multiple choice or in an open text field.

Question 10 – Prolog concepts resit 1 – 212525.4.0

Explain the meaning of *incompleteness* in Prolog.

Grading instruction

Correct answer (Number of points: 2)

Incompleteness is a property of Prolog proof search: Prolog does not always find a derivation, even if a derivation exists. (The answer should mention derivation or an equivalent).

Question 11 – Prolog_concepts1 – 211796.4.0

Which of the following phenomena are characteristic of Prolog?

- ☐ **A** Proof search does not always find a derivation, even if derivation exists.
- ☐ **B** The declarative and procedural meaning of a Prolog program are aligned.
- ☐ **C** Prolog always uses an occurs check.
- ☐ **D** Prolog searches the program in a depth first search fashion.

Question 12 – Prolog_concepts2 – 211812.1.0

Which of the following prolog predicates is defined in a tail-recursive way?

- ☐ **A** `xyz([],0).`
`xyz([_|L],NL) :- xyz(L,L2), NL is L2 + 1.`
- ☐ **B** `xyz([ ],X,X).`
`xyz([_|L],OX,L) :- NX is OX + 1, xyz(L,NX,L).`
- ☐ **C** `xyz([ ],X,X).` `xyz(X,[ ],X).`

Programming in Prolog: tf-idf

Question order: Fixed

A traditional metric in the field of information retrieval is *term frequency-inverse document frequency*, by which the importance of a word in a set of documents can be scored.

Given is the following set of documents, represented as a list of lists where each list represents one (short) document:

```
docs([
  [i,scream,you,scream,we,all,scream,for,ice,cream],
  [fuzzy,wuzzy,was,a,bear,fuzzy,wuzzy,had,no,hair,fuzzy,wuzzy,was,not,fuzzy,was,he],
  [i,wish,to,wash,my,irish,wrstwatch]
]).
```

You will implement the tf-idf metric in prolog in several steps, to be applied on the given list of lists.

Question 13 – 22-23_prolog_termfreq – 346536.3.0

Step 1: term frequency

The term frequency is calculated by simply counting the number of times a given word occurs in the complete set of documents.

Implement this metric in prolog, without using any helper predicates, such that it leads to the following outcomes:

```
?- docs(L), term_freq(i,L,T).
L = [[i, scream, you, scream, we, all, scream, for|...], [fuzzy, wuzzy, was, a, bear, fuzzy, wuzzy|...], [i,
wish, to, wash, my, irish|...]],
T = 2 .
?- docs(L), term_freq(scream,L,T).
L = [[i, scream, you, scream, we, all, scream, for|...], [fuzzy, wuzzy, was, a, bear, fuzzy, wuzzy|...], [i,
wish, to, wash, my, irish|...]],
T = 3 .
?- docs(L), term_freq(fuzzy,L,T).
L = [[i, scream, you, scream, we, all, scream, for|...], [fuzzy, wuzzy, was, a, bear, fuzzy, wuzzy|...], [i,
wish, to, wash, my, irish|...]],
T = 4 .
```

Note that the 'docs(L)' part of the query is needed to instantiate the variable L with our given set of documents.

Hint: it is easiest to write a separate predicate that counts the frequency of a word in a single list, and make use of this predicate in the term_freq/3 predicate by iterating over the separate lists.

Grading instruction

Correct outcome (Number of points: 4)

Example coding:

```
term_freq_list(X,[],0).
term_freq_list(X,[X|T],TF) :- term_freq_list(X,T,TFNew), TF is TFNew + 1.
term_freq_list(X,[Y|T],TF) :- not(X=Y), term_freq_list(X,T,TF).

term_freq(X,[],0).
term_freq(X,[H|T],TF) :- term_freq_list(X,H,TFL), term_freq(X,T,TFNew), TF is TFNew + TFL.
```

Both left-recursion and tail-recursion are fine.

Be a bit lenient on the not(X=Y) part

Question 14 – 22-23_prolog_inverse_document_frequency – 348072.3.1

Step 2: inverse document frequency

The second step is to calculate the inverse document frequency, which indicates how common or rare a word is in terms of the number of documents it occurs in:

$$\text{idf}(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|}$$

N = the total number of documents in the corpus D .

$|\{d \in D : t \in d\}|$ = the number of documents in which the term t appears.

Implement this metric as *idf*/3 in prolog, such that it leads to the following outcomes:

```
?- docs(L), idf(i,L,T).
L = [[i, scream, you, scream, we, all, scream, for|...], [fuzzy, wuzzy, was, a, bear, fuzzy, wuzzy|...], [i,
wish, to, wash, my, irish|...]],
T = 0.4054651081081644 .
?- docs(L), idf(scream,L,T).
L = [[i, scream, you, scream, we, all, scream, for|...], [fuzzy, wuzzy, was, a, bear, fuzzy, wuzzy|...], [i,
wish, to, wash, my, irish|...]],
T = 1.0986122886681098 .
?- docs(L), idf(fuzzy,L,T).
L = [[i, scream, you, scream, we, all, scream, for|...], [fuzzy, wuzzy, was, a, bear, fuzzy, wuzzy|...], [i,
wish, to, wash, my, irish|...]],
T = 1.0986122886681098 .
```

You may use the *member*/2 predicate, which returns true if a single term is part of a list:

```
?- member(a,[a,b,c,d]).
```

```
:- member(a11,[we,a11]).  
true.
```

Hint: it is easiest to write a separate predicate that counts the number of lists in which a given term occurs

The logarithmic calculation in this formula can be performed with the built-in 'log' operator in prolog:

```
?- X is log(10).  
X = 2.302585092994046.
```

Grading instruction

Correct implementation (Number of points: 5)

```
doc_freq(X, [], 0).  
doc_freq(X, [L|T], DF) :- member(X, L), doc_freq(X, T, DFNew), DF is DFNew+1.  
doc_freq(X, [L|T], DF) :- not(member(X, L)), doc_freq(X, T, DF).
```

```
idf(X, L, IDF) :- doc_freq(X, L, DF), length(L, Len), IDF is log(Len / DF).
```

penalize a point for particular aspects that are omitted, such as an additional rule body for lists where a term is not a member.

Also fine if a student chooses to perform tail-recursion.

Question 15 – 22-23 prolog tfidf – 348360.1.0

Step 3: term frequency - inverse document frequency

The tf-idf is finally calculated by simply multiplying the term frequency with the inverse document frequency, such that it returns the following outcomes:

```
?- docs(L), tfidf(i, L, T).  
L = [[i, scream, you, scream, we, all, scream, for|...], [fuzzy, wuzzy, was, a, bear, fuzzy, wuzzy|...], [i,  
wish, to, wash, my, irish|...]],  
T = 0.8109302162163288 .
```

```
?- docs(L), tfidf(scream, L, T).  
L = [[i, scream, you, scream, we, all, scream, for|...], [fuzzy, wuzzy, was, a, bear, fuzzy, wuzzy|...], [i,  
wish, to, wash, my, irish|...]],  
T = 3.295836866004329 .
```

```
?- docs(L), tfidf(fuzzy, L, T).  
L = [[i, scream, you, scream, we, all, scream, for|...], [fuzzy, wuzzy, was, a, bear, fuzzy, wuzzy|...], [i,  
wish, to, wash, my, irish|...]],  
T = 4.394449154672439 .
```

Write the tfidf/3 predicate by making use of the two predicates that you implemented in step 1 and 2 (*term_freq/3* and *idf/3*).

Hint: no need to include your particular implementation of these steps in your answer - you can simply assume that they lead to the proper outcome. Hence, you can score points also if you are not certain about your implementation of these steps.

Grading instruction

Correct implementation (Number of points: 2)

```
tfidf(X, L, TFIDF) :- term_freq(X, L, TF), idf(X, L, IDF), TFIDF is TF * IDF.
```

Question 16 – 22_23_prolog_programming_multiply – 313988.2.0

Which of the rules below is a proper specification of a predicate `multiply(L, M)`, that is true when `M` matches the multiplication of each term in `L`:

```
?- multiply([1,5,3], M).  
M = 15.
```

- A** `multiply([X], X).`
`multiply([H|T], M) :- multiply(T, MNew), M is MNew * H.`
- B** `multiply([], 0).`
`multiply([H|T], M) :- multiply(T, MNew), M is MNew * H.`
- C** `multiply([X], X).`
`multiply([H|T], M) :- multiply(T, M), MNew is M * H.`
- D** `multiply([], 0).`
`multiply([H|T], M) :- multiply(T, MNew), M is MNew * T.`
- E** `multiply([X], X).`
`multiply([H|T], M) :- multiply(T, MNew), M is MNew * T.`

Agents and MARBEL: basics

Question order: Fixed

This block features open text questions about knowledge on basic Agents and MARBEL.

Question 17 – 22-23_MARBEL_Basics1a – 349462.1.0

Multi-agent environments can be classied according to different properties.

Explain what it means for an environment to be *fully/partially* observable.

Grading instruction

Criterion 1 (Number of points: 1)

An agent can (not) always "see" everything...

Criterion 2 (Number of points: 1)

...relevant for its decision making.

Question 18 – 22-23_MARBEL_Basics1b – 349465.1.0

Apart from full/partial observability, mention two further properties that environments can have which are relevant for designing agents.

Grading instruction

Criterion 1 (Number of points: 1)

One of:

static/dynamic
discrete/continuous
single/multi agents
deterministic/stochastic

Criterion 2 (Number of points: 1)

Another one of:

static/dynamic
discrete/continuous
single/multi agents
deterministic/stochastic

Question 19 – 22-23_MARBEL_Basics2 – 349466.1.0

An agent does not always have full control over what happens in an environment and therefore the agent can not always be sure what the effects of performing an action are.

Mention two more specific reasons that explain why an agent might need to monitor what happens in the environment when it performs an action.

Grading instruction

Criterion 1 (Number of points: 2)

One of:

- (1) environment is dynamic (changes happen because of the environment)
- (2) environment is stochastic (agent cannot know outcome of stochastic effects of actions)
- (3) environment is multi-agent (there are other agents that make may interfere with or obstruct/block the actions of the agent)
- (4) (durative) actions may fail (this is very generic but OK too).

Criterion 2 (Number of points: 2)

Another one of:

- (1) environment is dynamic (changes happen because of the environment)
- (2) environment is stochastic (agent cannot know outcome of stochastic effects of actions)
- (3) environment is multi-agent (there are other agents that make may interfere with or obstruct/block the actions of the agent)
- (4) (durative) actions may fail (this is very generic but OK too)

(4) (durative) actions may fail (this is very generic but OK too).

Programming in MARBEL

Question order: Fixed

This assignment is about a (toy) coffee robot that is deployed in an office space with 5 offices. The robot is able to get coffee from the single coffee machine which is located in the public area of the office space and deliver it from there to any of the 5 offices. The robot can always move freely from one place to another in the office space. Carefully analyze the MARBEL agent system in the figure below. Make sure to follow the instructions given in Remark 1 of the figure. It is advisable to keep this block visible when answering the questions that follow.

Remark 1: Important for the MARBEL questions: When you add code in the program templates, clearly specify the position in the code where you want to add something.

For example, if you want to *modify* some module file's line 11, use the following notation:

file.mod2g 11 ...put modified code here....

If you want to place code in some module file between two line numbers, e.g. 23 and 24, use the following notation:

file.mod2g 23.1 ...new code line 1....

file.mod2g 23.2 ...new code line 2....

file.mod2g 23.3 ...new code line 3....

```
----- deliveryRobot.mas2g -----
1 use "sensors-and-actuators.jar".
2
3 define deliveryRobot {
4   use deliveryInit for init.
5   use delivery for decisions.
6
7   ...
8   ...
9 }
10
11 launchpolicy {
12   when * launch deliveryRobot.
13 }
-----
----- coffee.pl -----
1 :- dynamic at/1, coffee/0, office/1.
2
3 inOffice :- ...
-----

----- deliveryInit.mod2g -----
1 use coffee.
2
3 module deliveryInit {
4   ...
5 }
-----
----- delivery.mod2g -----
1 use coffee.
2
3 module delivery {
4   % If robot can deliver coffee, do it.
5   if delivery(Office) then deliverCoffee.
6   % Go to office to deliver coffee, if robot has coffee.
7   if delivery(Office), has(coffee) then move(Office).
8   % If robot needs to get coffee, get it.
9   if delivery(Office) then getCoffee.
10 }
```

Question 20 – 22-23_MARBEL_Programming1 – 349468.1.0

The coffee machine in the public office space area is also referred to as 'office area 0'. Add an action rule to the code in the most appropriate place (indicate in which line) to make sure that the agent knows this public office space area exists, i.e. `office(0)` is in its database.

Also add an action rule to add all other office areas to the agent's database using the `office/1` predicate.

Grading instruction

Criterion 1 (Number of points: 5)

1. In `deliveryInit.mod2g`, add anywhere, for example:
`deliveryInit.mod2g 4 if true then insert(office(0)).`
2. In `deliveryInit.mod2g`, add anywhere, for example:
`deliveryInit.mod2g 5 if true then insert(office(0), office(1), office(2), office(3),office(4), office(5)).`

It is always fine to use `forall-do` instead of `if-then`. If some mistake(s) in rule but more or less still ok, deduct 1pt. 1pt for putting rules in correct place, i.e. the init module. Also ok to use only a single rule, such as the second one above.

Question 21 – 22-23_MARBEL_Programming2 – 349477.1.0

Assume that `delivery/1` is a send always percept informing the agent that when it receives, for example, `delivery(2)`, someone in office 2 wants a coffee. Add a channel for handling this percept with the appropriate handler to the code and indicate where you would add this.

What else needs to be added to the program to make it work without problems (again indicate also where)?

Grading instruction

Criterion 1 (Number of points: 5)

1. In `deliveryRobot.mas2g`, line 8 or 9 put:
`replace delivery/1.`
2. The predicate needs to be declared. In `coffee.pl`, add to line 1:
`delivery/1.`

Correct place `mas2g` (1p), correct line (1p), and correct handler 'replace' (1p). Indication that `delivery/1` predicate needs to be declared (1p) and this needs to be added to line 1 in `coffee.pl` (1p). Just the phrase "In `coffee.pl`, add to line: `delivery/1`" is also OK (=2p).

Question 22 – 22-23_MARBEL_Programming3 – 349479.1.0

The `at/1` is a send on change percept informing the agent about whether it is at a particular office area. Add an appropriate handler for this percept and indicate where this should be put in the code.

Grading instruction

Criterion 1 (Number of points: 3)

In `deliveryRobot.mas2g`, line 8 or 9 put:
`update at/1.`

See for point distribution previous question part.

Question 23 – 22-23_MARBEL_Programming4 – 349480.1.0

The agent can only deliver coffee in an office but not when it is at the coffee machine in the public office area. The `inOffice` predicate has not been defined yet in the Prolog file `delivery.pl`, see line 2.

Provide a rule that defines this predicate and indicates the robot is in an office.

Grading instruction

Criterion 1 (Number of points: 2)

```
inOffice :- not(at(0)).
```

Deduct 1pt if a (minor) mistake is made.

Question 24 – 22-23_MARBEL_Programming5 – 349482.1.0

The deliveryRobot agent can only get coffee if it is at the coffee machine using the `getCoffee` action. Assume this action is instantaneous and the robot has coffee when it performs this action. In other words, no percept is needed (and also not received), but the agent needs to keep track of whether the agent has coffee itself.

Modify the program to make sure that the agent only performs the `getCoffee` action when it is at the coffee machine in the public office area and the query `has(coffee)` succeeds when the agent has just performed the `getCoffee` action.

Grading instruction

Criterion 1 (Number of points: 4)

Add `at(0)` to the query of the rule at line 9 in `delivery.mod2g`. Add `+ insert(has(coffee))` to the action part of the same rule.

1pt for place to modify (line 9, `delivery.mod2g`), 1pt for indicating that `at(0)` needs to be added to the query of this rule, 1pt for indicating that `insert(has(coffee))` needs to be added to the action part of the rule, and 1pt for using the `+` operator.

Question 25 – 22023_MARBEL_Programming6 – 349564.1.0

Why do we not have to add `not(has(coffee))` to the query of the rule in line 9 in `delivery.mod2g`?

Explain this by reasoning about the other rules that are in this module.

Grading instruction

Criterion 1 (Number of points: 2)

The rule at line 7 already checks for `has(coffee)`. Only if that rule is not applicable, the rule at line 9 would be evaluated.

Question 26 – 22-23_MARBEL_Programming7 – 349569.1.0

The deliveryRobot agent should only deliver coffee when it is at an office space where coffee is requested. In particular, it should not deliver any coffee in the public space area. Right after performing the action `deliverCoffee` (instantaneously) the robot does not have coffee any more.

Modify the program to make sure that the robot only delivers coffee when it is at an office where coffee has been requested and no longer thinks it has coffee.

Grading instruction

Criterion 1 (Number of points: 4)

Add `inOffice` to query in rule at line 5 in `delivery.mod2g` and add `+ delete(has(coffee))` to the action part of that rule.

See for distribution of points previous question part.

Question 27 – 22-23_MARBEL_Programming8 – 349570.1.0

The delivery robot does nothing if it does not have a delivery goal. Add a rule of the form `if true then ...` in `delivery.mod2g` that selects the move action to go back to the coffee machine in the right place in the program for handling that case.

Grading instruction

Criterion 1 (Number of points: 4)

Add `if true then move(0)` . at line 10.1 in `delivery.mod2g`.

2pts for adding the correct action and 2pts for the correct place.